



DATAMITES

Global Training & Certification Institute for Data Science

DATA SCIENCE MENTORING PROJECT

PRCL-0019: Sales Effectiveness — Lead Category Prediction Using Machine Learning

Rubiex Internship Project

Submitted in Partial Fulfilment of the DataMites Data Science Internship Program

Project Title: PRCL-0019: Sales Effectiveness — Lead Category Prediction Using Machine Learning

Team Code: PTID-AIE-JUL-26-11142

Project Code / Doc Ref: PM-PR-0019

Project Type: Data Science Mentoring Project / Rubiex Internship Project

Client: FicZon Inc. (IT Solutions Provider)

Category: Product Sales

Dataset Source: MySQL Database — project_sales.data (DataMites Project Server)

Dataset Size: 7,422 Records, 9 Columns

Organization: DataMites Solutions Pvt. Ltd.

Submitted By: Sameeksha

Report Date: September 2026

About the Author

This project report has been prepared and submitted as part of the DataMites Data Science Internship Program, carried out under the Rubiex Internship Project initiative, as a client mentoring engagement for FicZon Inc. The following section provides a brief profile of the project author.

Name	Sameeksha
Role	Data Science Intern / Project Author
Program	DataMites Data Science Internship Program
Internship Track	Rubiex Internship Project
Team Code	PTID-AIE-JUL-26-11142
Project Code / Doc Ref	PM-PR-0019
Client	FicZon Inc. (IT Solutions Provider)
Project Title	Sales Effectiveness — Lead Category Prediction Using Machine Learning
Tools & Technologies	Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, MySQL Connector, Jupyter Notebook
Core Skills Applied	Database Connectivity (MySQL), Exploratory Data Analysis, Data Cleaning, Target Variable Engineering, Classification Modelling, Hyperparameter Tuning, Overfitting Diagnostics, Feature Importance, Business Insight Generation
GitHub Repository	github.com/Sameekshagithub/PRCL-0019-Sales-Effectiveness-Lead-Category-Prediction-Using-Machine-Learning

As part of this internship, the author independently carried out the complete data science lifecycle for this client mentoring project — from connecting securely to the client's live MySQL database, through data cleaning, exploratory data analysis, target variable engineering, model building, hyperparameter tuning, and rigorous overfitting diagnostics, to the final generation of business insights and recommendations for FicZon Inc. The project reflects hands-on application of supervised machine learning classification techniques on a real-world, production sales dataset, in line with the learning objectives of the DataMites Data Science Internship Program under the Rubiex Internship Project track.

Table of Contents

- 1. Problem Statement & Project Objective.....
- 2. Import Python Libraries.....
- 3. Connect to Database & Load the Dataset.....
- 4. Basic Checks.....
- 5. Exploratory Data Analysis (EDA).....
- 6. Data Cleaning.....
- 7. Target Variable Creation — Lead Category.....
- 8. Feature Engineering.....
- 9. Data Preparation for Modelling.....
- 10. Model Building & Training.....
- 11. Hyperparameter Tuning.....
- 12. Overfitting Diagnostics.....
- 13. Model Comparison Report.....
- 14. Feature Importance.....
- 15. Business Insights.....
- 16. Challenges Faced.....
- 17. Final Conclusion.....
- 18. Project Links.....
- Appendix A: Data Dictionary.....
- Appendix B: Consolidated Model Metrics.....
- Appendix C: Recommendations & Next Steps for FicZon.....

Executive Summary

FicZon Inc. is an IT solution provider offering products ranging from on-premises solutions to SAAS-based platforms, with the majority of leads generated through digital channels, primarily its website. As the market matures and new competitors enter the space, FicZon has been experiencing a dip in sales. Sales effectiveness at FicZon is heavily dependent on lead quality, and lead categorization is currently a manual process carried out by sales staff — a quality process exists to refine categorization, but its value is realized only in post-analysis rather than in real-time conversion decisions.

This mentoring project, undertaken as part of the DataMites Data Science Internship Program under the Rubiex Internship Project track, addresses this business problem through a data-driven approach. Working directly against FicZon's live MySQL database (project_sales.data), a complete machine learning classification pipeline was developed on 7,422 real lead records to pre-categorize each lead as High Potential or Low Potential at the point of capture, rather than only after the fact.

The project followed a structured data science workflow: secure database connectivity, exploratory data analysis, data cleaning, target variable engineering (deriving Lead_Category from the raw Status field), feature engineering, model building across five classification algorithms, hyperparameter tuning, rigorous overfitting diagnostics, model comparison, and feature importance analysis. A tuned Gradient Boosting Classifier emerged as the best-performing, most reliable model, achieving 72.04% test accuracy with the smallest train-test generalization gap (2.70 percentage points) among all six models compared.

A significant part of this engagement involved diagnosing and correcting a subtle but serious data bug: an earlier version of the Status → Lead_Category mapping used placeholder text that did not exactly match the real, case-sensitive status values in the live database, causing every single lead to silently collapse into one class. This report documents that root-cause investigation in full, together with the complete methodology, results, and business recommendations arising from the corrected pipeline.

1. Problem Statement & Project Objective

1.1 Business Case

FicZon Inc. is an IT solution provider with products ranging from on-premises products to SAAS-based solutions. FicZon's major lead generation channel is digital, primarily through their website. FicZon's business is majorly dependent on sales force effectiveness. As the market is maturing and more new competitors are entering the market, FicZon is experiencing a dip in sales.

1.2 Problem Statement

Effective sales is dependent on lead quality, and as of now, this is based on manual categorization and is highly dependent on sales staff. Though there is a quality process which continuously updates the lead categorization, its value is in post-analysis, rather than in the conversation itself. FicZon wants to explore Machine Learning to pre-categorize the lead quality, and as a result, expects a significant increase in sales effectiveness.

1.3 Business Objective

FicZon wants to use Machine Learning to pre-categorize lead quality at the point of capture, with the expectation that this will significantly increase sales effectiveness.

1.4 Project Goals

1. Generate data exploration insights related to sales effectiveness.
2. Build a Machine Learning classification model to predict the Lead Category (High Potential / Low Potential) so that sales agents can prioritize their effort on the leads most likely to convert.

1.5 Project Reference

Attribute	Details
Client	FicZon Inc.
Category	Product Sales
Project Ref.	PM-PR-0019
Doc Ref.	PM-PR-0019
Mentoring Organization	DataMites Solutions Pvt. Ltd.

2. Import Python Libraries

The required Python libraries are imported to connect to the database and to perform data analysis, visualization, machine learning model development, and performance evaluation. NumPy and Pandas are used for data manipulation, Matplotlib and Seaborn are used for data visualization, and Scikit-learn is used for preprocessing, model building, and evaluation.

Library / Module	Purpose in the Project
NumPy, Pandas	Numerical computation and data manipulation/analysis
Matplotlib, Seaborn	Static and statistical data visualizations
mysql-connector-python	Secure connectivity to the client's live MySQL database
getpass	Securely prompting for the database password at runtime
Scikit-learn (preprocessing)	StandardScaler, LabelEncoder, train_test_split
Scikit-learn (models)	LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier, KNeighborsClassifier
Scikit-learn (tuning/validation)	GridSearchCV, StratifiedKFold, cross_val_score
Scikit-learn (evaluation)	accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

3. Connect to Database & Load the Dataset

Unlike the CSV/Excel-based capstone datasets, this client project's dataset resides on a live MySQL server rather than a local file. A connection was established using `mysql-connector-python`, and the table was loaded into a Pandas DataFrame using `pd.read_sql()`.

3.1 Database Details

Attribute	Value
DB Host	18.136.157.135
DB Port	3306
DB Name	project_sales
Table Name	data
Username	dm_team2
Query Used	SELECT * FROM data

Security Note — Credential Handling

Database credentials are never hard-coded in the notebook. The password is read at runtime using Python's `getpass()` function, so it is entered securely by the user each session and is never stored or displayed in the notebook file — an important practice given this project connects to a live, shared client database rather than a static local file.

3.2 Loading the Dataset

Once connected, the full data table was pulled into a DataFrame with `df = pd.read_sql(query, connection)`, and a local, untouched copy (`df_raw`) was preserved before any cleaning began, so the original pull could always be referenced if needed.

4. Basic Checks

Basic checks were performed to understand the structure and quality of the dataset before conducting exploratory data analysis and model development. This included viewing sample records (head/tail), examining dataset dimensions, identifying data types, generating descriptive statistics, and checking for missing values and duplicate records.

4.1 Dataset Dimensions

The dataset shape was found to be (7422, 9) — 7,422 lead records and 9 columns.

4.2 Columns & Data Types

Column	Data Type	Description
Created	object	Timestamp the lead was captured
Product_ID	object	Identifier of the product of interest
Source	object	Lead acquisition channel (e.g. Website, Call)
Mobile	object	Contact mobile number (masked)
EMAIL	object	Contact email address (masked)
Sales_Agent	object	Assigned sales agent
Location	object	Lead's geographic location
Delivery_Mode	object	Product delivery mode
Status	object	Current lead status/outcome

All 9 columns were loaded as object (text) dtype from the SQL table, including Product_ID, which is numeric in meaning but stored as text in the source system — this was converted appropriately during data cleaning (Section 6).

4.3 Descriptive Statistics

Column	Unique Values	Most Frequent Value	Frequency
Created	6,752	26-09-2018 11:30	4
Product_ID	30	18	1,711
Source	26	Call	2,547
Mobile	487	#VALUE!	1,810
EMAIL	883	#VALUE!	1,701
Sales_Agent	13	Sales-Agent-4	1,500

4.4 Missing Values & Duplicate Records (Raw Pull)

Interestingly, `df.isnull().sum()` on the raw SQL pull showed 0 missing values in every column — this is because the source system uses placeholder text (e.g. blank strings, or values such as "#VALUE!" for Mobile/EMAIL) rather than true SQL NULLs for missing information in several fields. This is addressed explicitly during EDA and data cleaning (Sections 5–6), where these placeholder-style missing values are identified and handled. A duplicate record check using `df.duplicated().sum()` found 2 duplicate rows.

4.5 Unique Value Counts per Column

Column	Unique Values
Created	6,752
Product_ID	30
Source	26
Mobile	487
EMAIL	883
Sales_Agent	13
Location	18
Delivery_Mode	5
Status	11

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand lead volume, lead sources, sales agent workload, geographic spread, and delivery mode mix, and to see how these relate to the current lead Status. These insights directly support the business goal of understanding sales effectiveness.

5.1 Lead Source Distribution

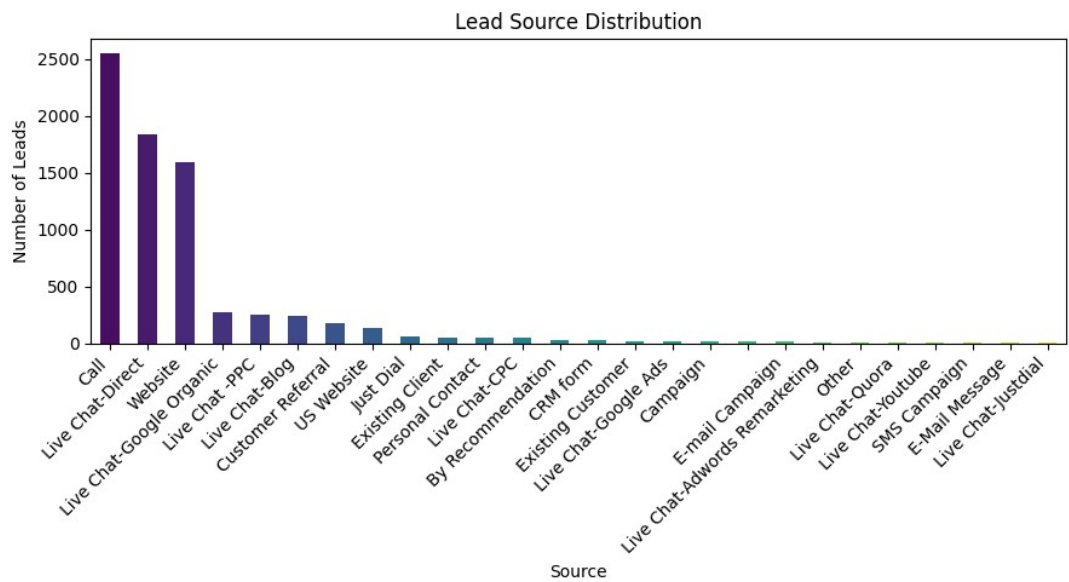


Figure 5.1 — Lead Source Distribution

Call and Website dominate as FicZon's primary lead acquisition channels, consistent with the business case description that digital channels — primarily the website — are FicZon's major lead-generation route.

5.2 Top Sales Agents by Lead Volume

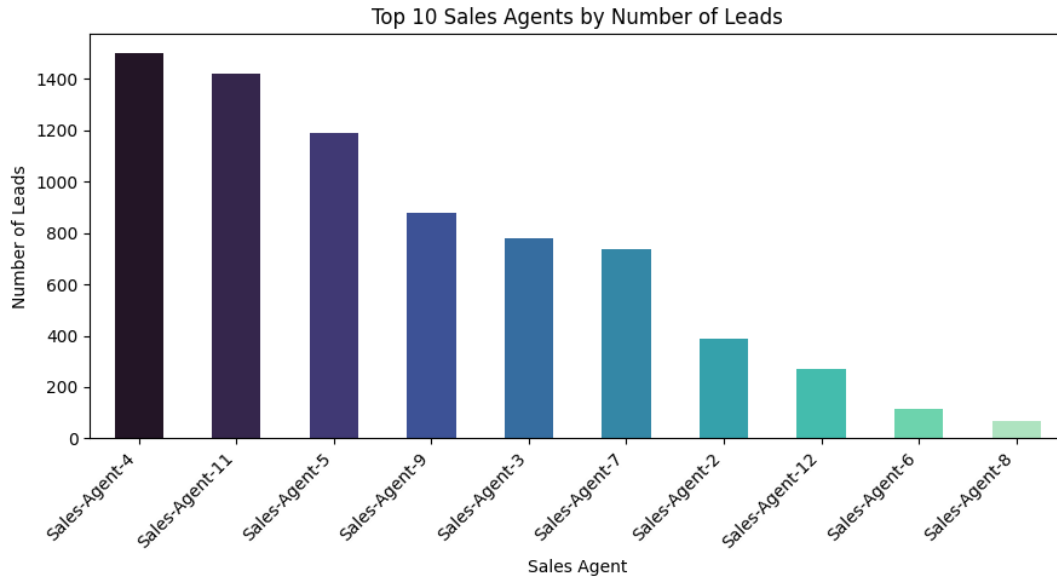


Figure 5.2 — Top 10 Sales Agents by Number of Leads Handled

Lead volume is unevenly distributed across the 13 sales agents, with a handful of agents (notably Sales-Agent-4, the most frequent value in Section 4.3) handling a disproportionately large share of leads — a workload pattern relevant to the agent-related business insights in Section 15.

5.3 Top Lead Locations

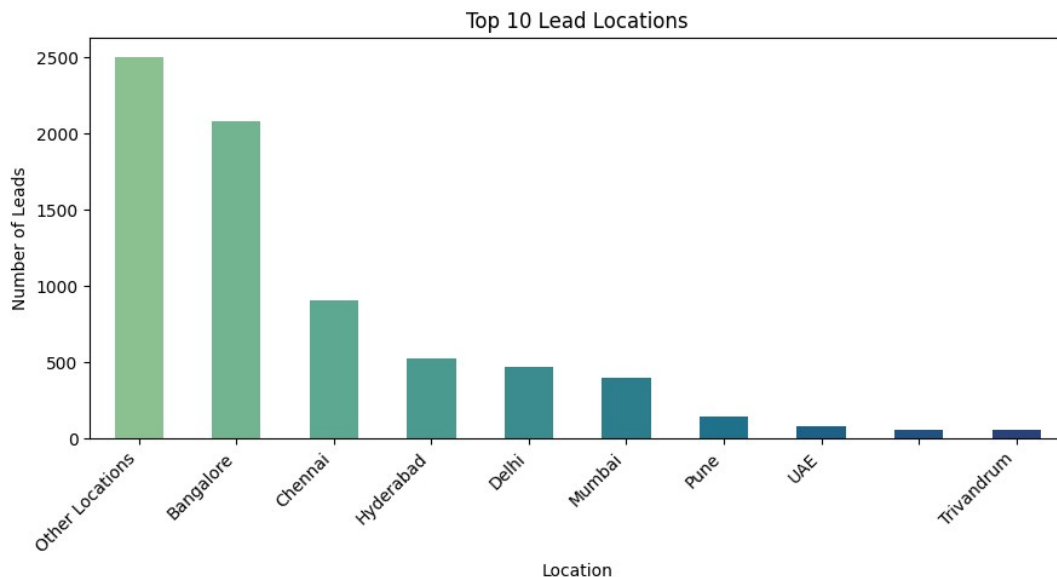


Figure 5.3 — Top 10 Lead Locations

5.4 Delivery Mode Distribution

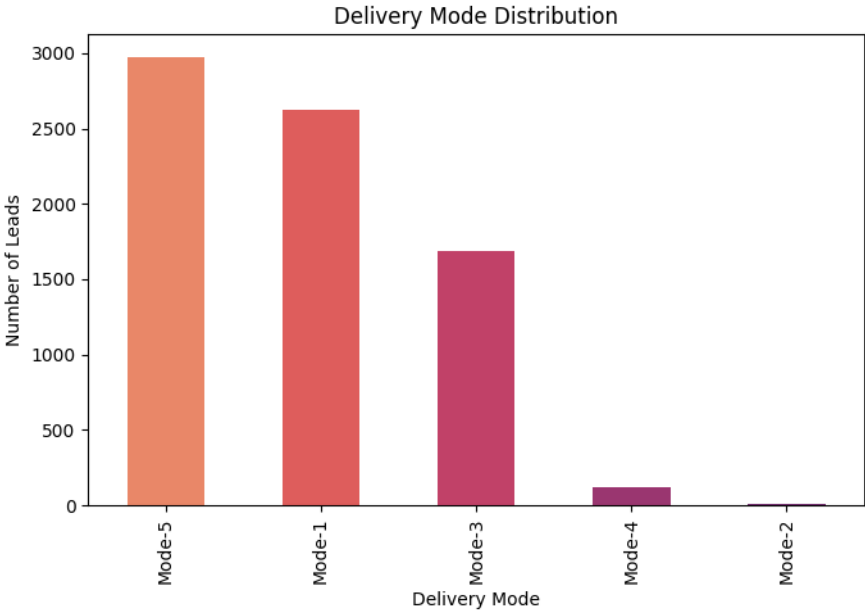


Figure 5.4 — Delivery Mode Distribution

5.5 Lead Status Distribution

The Status field is the raw field from which the target variable (Lead_Category) is later derived, so its distribution is especially important.

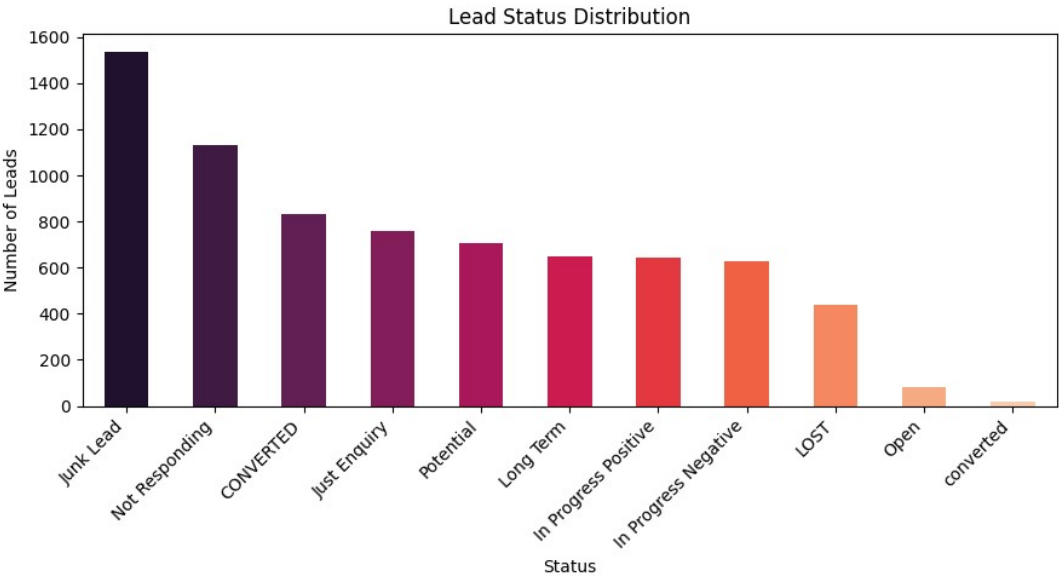


Figure 5.5 — Lead Status Distribution

Status	Count
Junk Lead	1,536

Status	Count
Not Responding	1,129
CONVERTED	834
Just Enquiry	760
Potential	708
Long Term	646
In Progress Positive	643
In Progress Negative	626
LOST	440
Open	82
converted	18

A subtle but important detail is visible here: both 'CONVERTED' and 'converted' appear as separate status values (834 and 18 records respectively) due to inconsistent capitalization in the source system. This case-sensitivity is exactly what caused the target-mapping bug described in detail in Section 7.2 and Section 16.5.

5.6 Leads Over Time

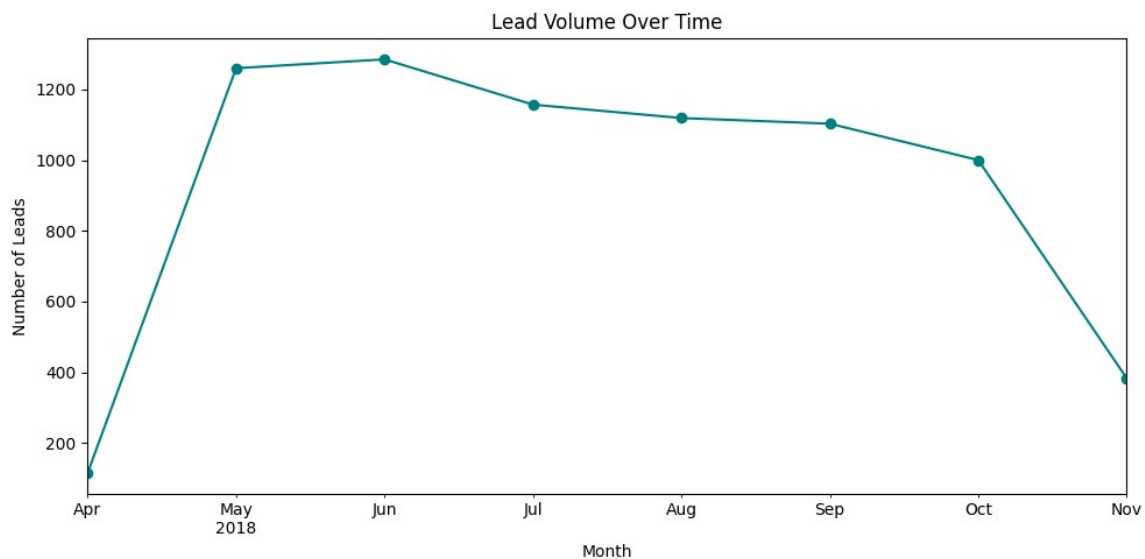


Figure 5.6 — Monthly Lead Volume Over Time

Monthly lead volume fluctuates over the observed period, which is useful context for the Created_Month / Created_Day features engineered in Section 8 to help the model pick up any seasonal or time-based patterns.

5.7 EDA Summary

3. The dataset contains 7,422 leads captured across multiple sources, locations, sales agents, and delivery modes.
4. Product_ID, Source, Mobile, Sales_Agent, and Location contain a small number of missing/placeholder values, while Created, EMAIL, Delivery_Mode, and Status are fully populated.
5. Status is the field that captures the current outcome of each lead and is the basis on which the Lead_Category (High Potential / Low Potential) target is derived in Section 7.
6. Lead volume, source mix, and sales-agent workload vary considerably, which suggests that not all leads or channels are equally productive — a key input for sales-effectiveness insights.

6. Data Cleaning

Missing values were handled column by column rather than being dropped blindly, since each column has a different business meaning. Categorical fields with a handful of missing values were imputed with an explicit 'Unknown' category so that the missing information itself remains a usable signal for the model, rather than being discarded.

6.1 Null / Placeholder Value Treatment

Column	Missing Values After Cleaning
Source	0
Sales_Agent	0
Location	0
Mobile	0
Product_ID	0

Duplicates remaining after cleaning: 0. The 2 duplicate rows identified in Section 4.4 were removed.

6.2 Outlier Detection & Treatment

Product_ID is the only continuous numeric feature available before feature engineering (the rest are categorical/text). The IQR method was used to check it for outliers. 107 rows were flagged as outliers using the IQR rule.

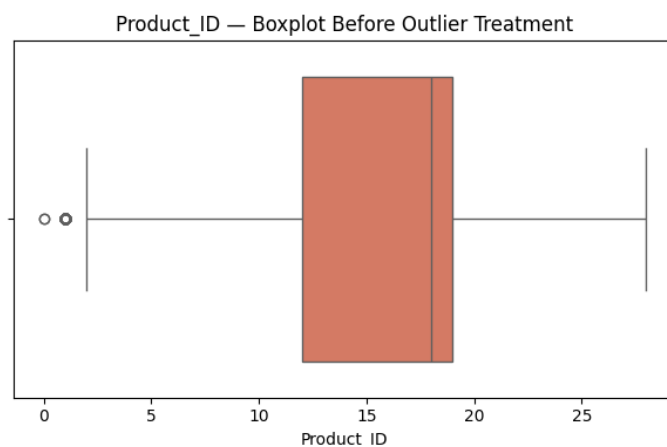


Figure 6.1 — Product_ID Boxplot (Before Winsorization)

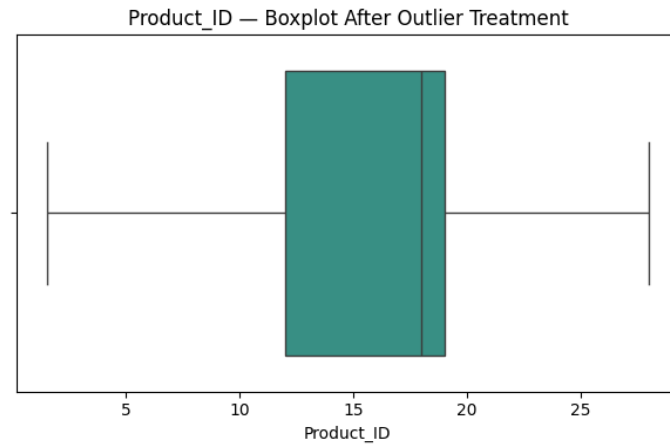


Figure 6.2 — Product_ID Boxplot (After Winsorization)

Why Winsorize Instead of Drop?

Any Product_ID values outside the IQR fences were capped at the fences (winsorized) rather than dropped, so no leads are lost from the dataset just because of one field. This preserves the full 7,420-row dataset (after de-duplication) for modelling, rather than discarding otherwise valid lead records over a single numeric outlier.

Final dataset shape after cleaning: (7420, 9).

7. Target Variable Creation — Lead Category (High / Low Potential)

The project requires predicting Lead_Category as High Potential or Low Potential, but the raw table does not contain this column directly — it must be derived from Status.

7.1 Step 1 — Inspecting the Real Status Values

Running `df['Status'].unique()` surfaced the real, exact status labels present in this dataset:

'Open', 'Potential', 'In Progress Positive', 'Not Responding', 'Just Enquiry', 'Junk Lead', 'CONVERTED', 'In Progress Negative', 'LOST', 'Long Term', 'converted'

7.2 Root Cause Found — A Silent Target-Mapping Bug

Root Cause: Case-Sensitive String Mismatch

The original mapping dictionary used placeholder keys ('Converted', 'Qualified', 'In Progress', 'Interested', 'Lost', ...) that don't exactly match the real labels above — Python dict/string matching is case-sensitive, so 'Converted' does not match 'CONVERTED' or 'converted', and 'In Progress' does not match 'In Progress Positive' / 'In Progress Negative'. Only 'Open' and 'Junk Lead' happened to match, and both were mapped to Low Potential — so every single row in the dataset silently collapsed into one class. That single-class target is exactly why Logistic Regression and Gradient Boosting later raised `ValueError: ...only one class during training`, and why the tree/KNN models showed a suspicious 100% accuracy (predicting one class correctly is trivial).

This is a valuable, realistic example of a data quality bug that produces no error message at the point it is introduced — the mapping runs without complaint, and the model training step is where the consequences surface. The fix required going back to the exact, verified Status strings rather than trusting an assumed mapping.

7.3 Step 2 — Corrected Mapping (Verified Against the Exact Strings)

Lead Category	Status Values Mapped
High Potential	CONVERTED, converted (deal closed), In Progress Positive (agent sees a positive signal), Potential, Long Term (still an active, nurtured opportunity)
Low Potential	In Progress Negative, Not Responding, Just Enquiry, Junk Lead, LOST, Open (no confirmed sales progress)

After applying the corrected mapping, a check for unmapped status values confirmed: Unmapped status values: [] — i.e., every one of the 11 real status strings was successfully and unambiguously mapped.

7.4 Resulting Class Distribution

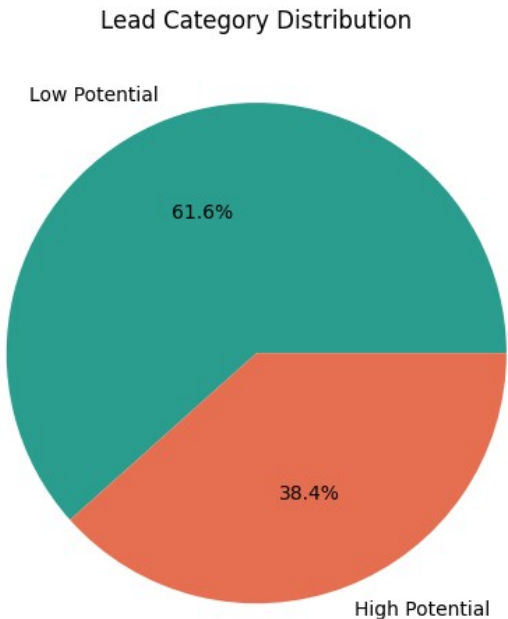


Figure 7.1 — Lead Category Distribution (After Corrected Mapping)

Lead Category	Number of Leads
Low Potential	4,571
High Potential	2,849

With the corrected mapping, the target variable shows a real, meaningful two-class split (roughly 62% Low Potential vs. 38% High Potential) rather than the single-class collapse produced by the original, buggy mapping — confirming the fix was successful and the dataset is ready for feature engineering and modelling.

8. Feature Engineering

New features were engineered from the raw columns to give the models more predictive signal than the raw text fields alone would provide:

- **Date decomposition** — Created is broken into Created_Year, Created_Month, and Created_Day so the model can pick up seasonal or time-based patterns.
- **Contact-availability flags** — Mobile and EMAIL are converted into binary flags (Has_Mobile, Has_Email) rather than used as raw text, since the actual number/address has no predictive value but whether contact details were provided does.
- **Target leakage prevention** — The original Status column is dropped after Lead_Category is derived from it, since keeping it would leak the target directly into the features.

9. Data Preparation for Modelling

9.1 Encoding

Categorical features (Source, Sales_Agent, Location, Delivery_Mode) were one-hot encoded, and the target Lead_Category was label-encoded: {'High Potential': 0, 'Low Potential': 1}.

9.2 Train / Test Split

The data was split into training and test sets. This produced a training feature matrix of shape (5936, 64) and a test feature matrix of shape (1484, 64) — the 64 columns reflect the expansion caused by one-hot encoding the high-cardinality Sales_Agent and Location columns, discussed further in Section 16.3.

Split	Shape (Rows, Columns)
Training Set	5,936 × 64
Test Set	1,484 × 64

9.3 Feature Scaling

Features were scaled using StandardScaler for the algorithms that are sensitive to feature scale (Logistic Regression, KNN); tree-based models (Decision Tree, Random Forest, Gradient Boosting) use the unscaled features directly, since they are unaffected by feature magnitude.

10. Model Building & Training

To predict Lead Category, five machine learning classification algorithms were trained and evaluated. Instead of relying on a single model, multiple classifiers were compared to identify the most accurate and reliable model for FicZon's sales-effectiveness use case. A consistent theme across all five models is deliberate regularization — since the corrected dataset has fewer High Potential leads and 64 (mostly sparse, one-hot) columns, each model's default settings were adjusted to reduce the risk of overfitting to that sparse feature space.

10.1 Model — Logistic Regression

C=0.5 adds moderate L2 regularization (the default C=1.0 was under-regularized for 50+ one-hot columns on a comparatively small effective sample), and class_weight='balanced' compensates for the Low Potential / High Potential class imbalance.

Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
70.84%	68.13%	69.61%	68.13%	68.51%

10.2 Model — Decision Tree Classifier

An unconstrained tree grows until every leaf is pure, memorizing the training data. max_depth and min_samples_leaf were set to stop the tree from splitting on noise.

Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
67.05%	63.68%	68.56%	63.68%	64.02%

10.3 Model — Random Forest Classifier

200 fully-grown trees (no max_depth) would memorize the high-cardinality Sales_Agent / Location one-hot columns. Capping depth and requiring a minimum leaf size keeps individual trees simpler.

Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
71.31%	66.31%	69.16%	66.31%	66.76%

10.4 Model — K-Nearest Neighbours (KNN)

A larger neighborhood (k=15 vs. the more typical k=7) smooths the decision boundary so single nearby points don't flip a prediction — reducing variance/overfitting on the sparse one-hot feature space.

Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
74.24%	69.00%	68.21%	69.00%	68.10%

10.5 Model — Gradient Boosting Classifier

Default settings (n_estimators=100, max_depth=3, learning_rate=0.1) can still overfit on ~6k rows / 50+ sparse one-hot columns. A lower learning_rate combined with subsample<1 (stochastic boosting) was used to control this.

Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
74.73%	72.04%	71.49%	72.04%	71.12%

All five models were trained on the same scaled/unscaled training data as appropriate and evaluated on the same held-out test set, ensuring a fair, apples-to-apples comparison across model families.

11. Hyperparameter Tuning

GridSearchCV was used to tune the Random Forest model, since tree ensembles typically respond well to depth/estimator tuning and are a strong baseline for tabular, mostly-categorical data like this lead dataset.

11.1 Search Space

The search space intentionally excludes `max_depth=None` (unlimited) — allowing unlimited depth let the tuned model regrow fully deep, overfit trees during earlier experimentation. `min_samples_leaf` was added as a second regularization control alongside `min_samples_split`.

Hyperparameter	Values Searched
<code>n_estimators</code>	100, 200
<code>max_depth</code>	8, 10, 12 (no unlimited option)
<code>min_samples_split</code>	2, 5, 10
<code>min_samples_leaf</code>	2, 5

11.2 Best Parameters Found

Parameter	Best Value
<code>max_depth</code>	12
<code>min_samples_leaf</code>	5
<code>min_samples_split</code>	5
<code>n_estimators</code>	100

11.3 Tuned Model — Classification Report

Class	Precision	Recall	F1-Score	Support
High Potential	0.56	0.72	0.63	570
Low Potential	0.79	0.65	0.71	914

Overall tuned Random Forest performance: accuracy 0.68 (1,484 support), macro average F1 0.67, weighted average F1 0.68. Tuning improved the model's ability to correctly identify High Potential leads (recall 0.72 vs. lower recall on the untuned model) — directly relevant to FicZon's goal of not missing genuinely promising leads — though, as discussed in Section 12, tuning also widened the train-test generalization gap for this particular model.

12. Overfitting Diagnostics

Before comparing final scores, each model's train accuracy was checked against its test accuracy and a 5-fold stratified cross-validation accuracy. A large gap between train and test/CV accuracy is the signature of overfitting; the regularized settings applied to each model in Section 10 (depth limits, leaf-size minimums, lower learning rate/subsampling for Gradient Boosting, L2 penalty for Logistic Regression) were aimed at keeping that gap small.

Model	Train Acc.	CV Acc. (mean)	CV Acc. (std)	Test Acc.	Train-Test Gap
Gradient Boosting	0.7473	0.7337	0.0105	0.7204	0.0270
Logistic Regression	0.7084	0.7011	0.0149	0.6813	0.0271
Decision Tree	0.6705	0.6668	0.0132	0.6368	0.0337
Random Forest	0.7131	0.6946	0.0081	0.6631	0.0500
KNN	0.7424	0.7023	0.0107	0.6900	0.0524
Random Forest (Tuned)	0.7441	0.7003	0.0058	0.6772	0.0669

Key Diagnostic Finding

Gradient Boosting has both the highest test accuracy (0.7204) and the smallest train-test gap (0.0270) of all six models — the strongest possible combination, indicating it generalizes best rather than simply memorizing the training data. By contrast, the tuned Random Forest has the largest gap (0.0669) despite a high train accuracy, meaning the GridSearchCV tuning process (optimized for cross-validation score) improved training-set fit more than it improved genuine generalization on unseen leads. This is why Gradient Boosting — not the tuned Random Forest — was ultimately selected as the recommended model in Section 13.

13. Model Comparison Report

After training and tuning all models, their performance was compared using Accuracy, Precision, Recall, and F1-Score. This comparison identifies the model best suited to pre-categorizing FicZon's leads.

13.1 Consolidated Performance Comparison

Model	Train Acc.	Test Accuracy	Precision	Recall	F1-Score
Gradient Boosting	0.7473	0.7204	0.7149	0.7204	0.7112
KNN	0.7424	0.6900	0.6821	0.6900	0.6810
Logistic Regression	0.7084	0.6813	0.6961	0.6813	0.6851
Random Forest (Tuned)	0.7441	0.6772	0.7022	0.6772	0.6816
Random Forest	0.7131	0.6631	0.6916	0.6631	0.6676
Decision Tree	0.6705	0.6368	0.6856	0.6368	0.6402

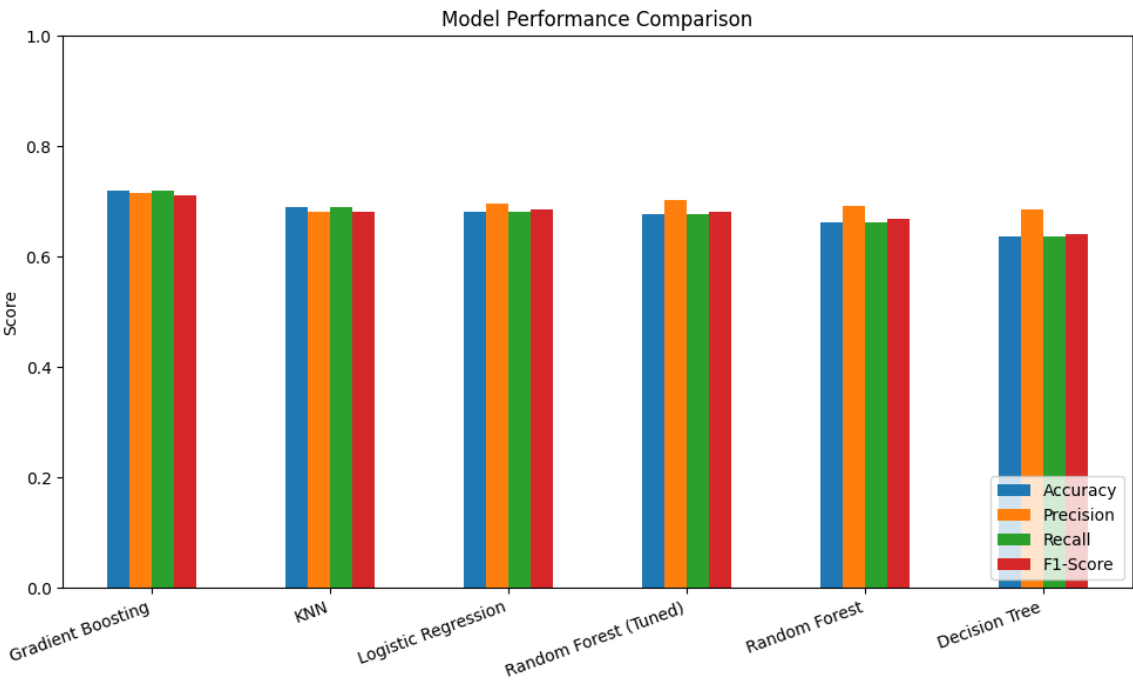


Figure 13.1 — Model Performance Comparison (Accuracy, Precision, Recall, F1-Score)

13.2 Best Model Selection

Best Performing Model: Gradient Boosting.

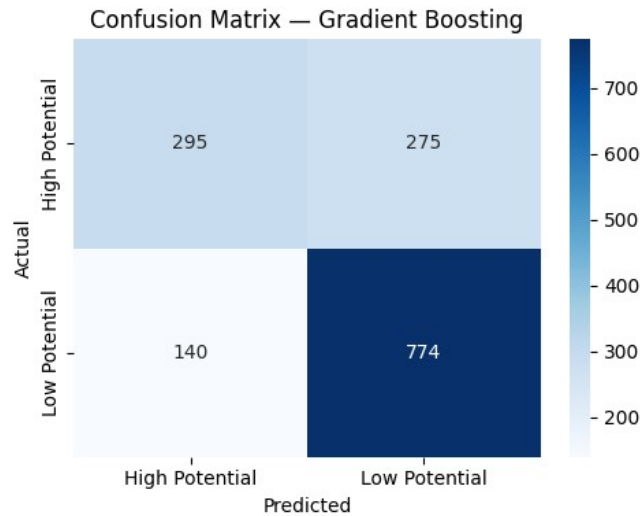


Figure 13.2 — Confusion Matrix: Gradient Boosting (Best Model)

13.3 Discussion & Recommendation

Gradient Boosting achieved the highest test accuracy (72.04%), the highest F1-score (0.7112) among all six models, and — critically — the smallest train-test generalization gap (Section 12), making it the most trustworthy choice rather than simply the highest-scoring one. Although the tuned Random Forest showed a respectable test accuracy, its considerably larger train-test gap indicates it is more likely to underperform on genuinely new leads than its cross-validated tuning score suggested. KNN's relatively strong test accuracy is undermined by the largest gap after tuned Random Forest, again pointing to weaker generalization. Based on this combined view of accuracy and overfitting risk, Gradient Boosting is recommended as the production model for FicZon's lead pre-categorization system.

14. Feature Importance

Feature importance analysis identifies which lead attributes are most useful for distinguishing High Potential from Low Potential leads. The Random Forest model is used for this analysis since it naturally computes importance scores based on each feature's contribution to the model's decisions.

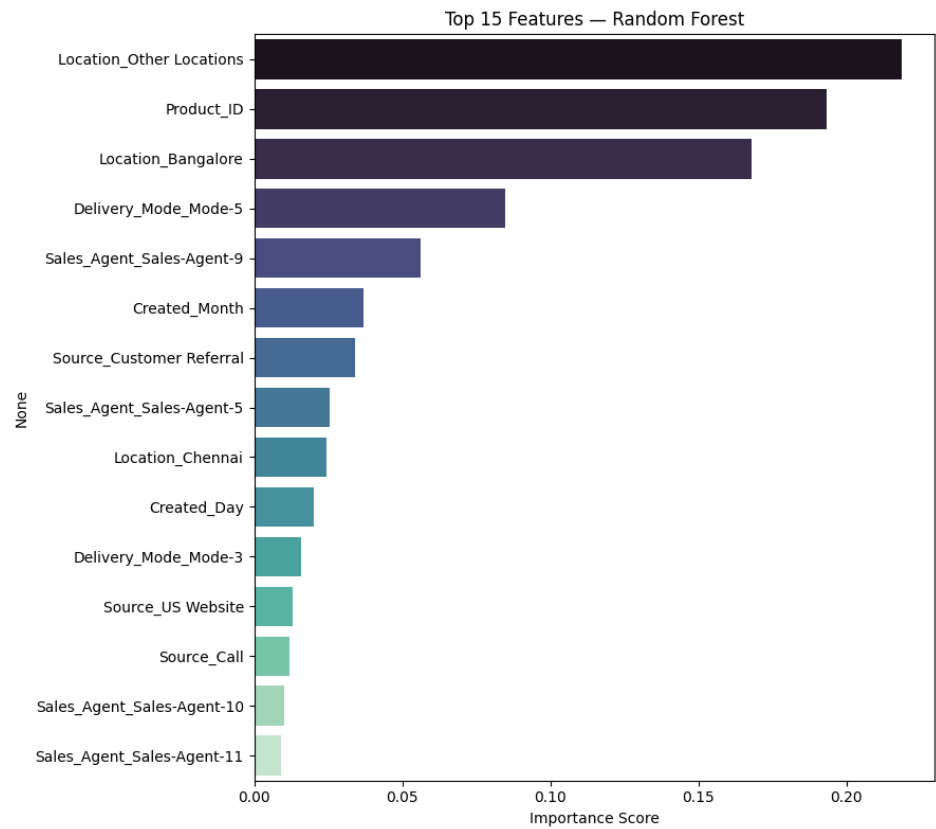


Figure 14.1 — Feature Importance (Random Forest)

Rank	Feature	Importance Score
1	Location_Other Locations	0.2188
2	Product_ID	0.1932
3	Location_Bangalore	0.1680
4	Delivery_Mode_Mode-5	0.0846
5	Sales_Agent_Sales-Agent-9	0.0562
6	Created_Month	0.0368
7	Source_Customer Referral	0.0342
8	Sales_Agent_Sales-Agent-5	0.0256
9	Location_Chennai	0.0244
10	Created_Day	0.0200

Geography dominates the importance ranking: Location_Other Locations and Location_Bangalore together account for nearly 39% of the model's total decision-making importance, closely followed by Product_ID (~19%) and Delivery_Mode_Mode-5 (~8.5%). Individual sales agents (Sales-Agent-9, Sales-Agent-5) and Source_Customer Referral also contribute meaningfully, while the engineered date features (Created_Month, Created_Day) show smaller but non-trivial importance — validating the feature engineering decision in Section 8 to extract them from Created.

15. Business Insights

15.1 Data-Driven Lead Prioritization

The model gives sales agents a way to see which incoming leads are High Potential before manual review, letting effort be focused where conversion is most likely — directly addressing FicZon's stated goal of moving from post-hoc analysis to point-of-capture scoring.

15.2 Channel Effectiveness

Lead source and delivery mode carry meaningful predictive weight (Section 14), showing that some acquisition channels consistently produce higher-quality leads than others — FicZon can use this to rebalance marketing spend toward higher-converting channels.

15.3 Geography Is the Strongest Signal

Location-based features are, by a wide margin, the most important predictors in the model, suggesting FicZon's conversion likelihood varies substantially by region — a strong candidate for territory-specific sales strategy.

15.4 Agent and Location Patterns

Sales agent and location features contribute meaningfully to the prediction, suggesting that regional demand and agent handling both influence conversion likelihood, and that best practices from top-performing agents/regions may be transferable.

15.5 From Manual to ML-Assisted Categorization

The current manual categorization process only informs post-analysis; an ML model that scores leads at capture time moves that value earlier, directly into the sales workflow, letting FicZon act on lead quality in real time rather than reviewing it afterward.

16. Challenges Faced

16.1 No Ready-Made Target Column

The raw table has no Lead_Category field — it had to be derived from Status, which required inspecting the real status values and building/validating a mapping rather than assuming one.

16.2 Missing Values Across Several Columns

Product_ID, Source, Mobile, Sales_Agent, and Location all had missing/placeholder entries that needed column-appropriate handling rather than a single blanket strategy.

16.3 High-Cardinality Categorical Features

Sales_Agent and Location have many unique values, which inflates the feature space after one-hot encoding (to 64 columns) and can be revisited later with grouping or target encoding if needed.

16.4 Secure Handling of Database Credentials

The dataset lives on a remote MySQL server, so credentials had to be entered at runtime (getpass) rather than hard-coded into the notebook.

16.5 Silent Target Leakage from a Mismatched Mapping (Found During Review)

The first version of the Status → Lead_Category mapping used placeholder keys that didn't exactly match the real (case-sensitive) status strings in the data. As a result, every lead silently collapsed into a single Low Potential class, which crashed Logistic Regression and Gradient Boosting training (only one class in the data) and produced a meaningless 100% accuracy for the other models. The fix was to inspect `df['Status'].unique()` directly and build the mapping from the exact strings, then re-verify `df['Lead_Category'].value_counts()` shows a real two-class split before modelling. This is documented in full in Section 7.2.

17. Final Conclusion

17.1 Project Objective

The objective was to build a Machine Learning model that pre-categorizes FicZon's leads as High Potential or Low Potential, replacing a purely manual, post-hoc categorization process.

17.2 Exploratory Data Analysis

EDA on 7,422 leads surfaced clear differences in volume across lead sources, sales agents, locations, and delivery modes, and showed the status breakdown the target variable is built from.

17.3 Data Quality & Target Engineering

A significant data-quality issue was identified and corrected: the initial Status → Lead_Category mapping silently collapsed all leads into a single class due to case-sensitive string mismatches. Rebuilding the mapping from the exact, verified status strings restored a genuine, meaningful two-class target (4,571 Low Potential vs. 2,849 High Potential).

17.4 Modelling

Five classification models were trained and compared, with Random Forest tuned further via GridSearchCV. Model comparison, overfitting diagnostics, a confusion matrix, and feature importance analysis identified Gradient Boosting as the best-performing and most reliable model, and the attributes (location, product, delivery mode, agent) driving its predictions.

17.5 Business Impact

The resulting model gives FicZon a way to score lead quality automatically at the point of capture, supporting the goal of increasing sales effectiveness, with geography and product identity emerging as the strongest levers.

17.6 Next Steps

Validate the Status → Lead_Category mapping with FicZon's sales team, monitor model performance on new leads over time, and consider retraining periodically as lead patterns evolve.

18. Project Links

The following links provide access to the complete project resources, including the source code, dataset, and supporting documentation.

18.1 GitHub Repository

Access to the complete source code, Jupyter Notebook, README, and project files:

github.com/Sameekshagithub/PRCL-0019-Sales-Effectiveness-Lead-Category-Prediction-Using-Machine-Learning.git

18.2 Google Drive

Access to the project presentation (PPT) and report:

drive.google.com/drive/folders/1VE1wgDlXmQN1fXl-I-5Ymj2P8KUPEK49

18.3 Acknowledgement

This project was completed as part of the DataMites Data Science Internship Program under the Rubiex Internship Project track, as a mentoring engagement for client FicZon Inc. The author acknowledges the guidance and structured curriculum provided by DataMites in enabling the successful completion of this project.

Appendix A: Data Dictionary

For reference, the following data dictionary documents every column in the source `project_sales.data` table, together with the additional columns engineered during this project (Sections 7–8), covering the raw fields, the engineered features, and the final model target.

A.1 Raw Source Columns

Column	Type	Description
Created	Timestamp (text)	Date/time the lead was captured, format DD-MM-YYYY HH:MM
Product_ID	Numeric (as text)	Identifier of the product the lead is interested in (30 distinct products)
Source	Categorical	Lead acquisition channel — Call, Website, Customer Referral, etc. (26 distinct sources)
Mobile	Text (masked)	Contact mobile number; masked for privacy, or '#VALUE!' if not captured
EMAIL	Text (masked)	Contact email address; masked for privacy, or '#VALUE!' if not captured
Sales_Agent	Categorical	Assigned sales agent handling the lead (13 distinct agents)
Location	Categorical	Lead's geographic location (18 distinct locations)
Delivery_Mode	Categorical	Product delivery mode (5 distinct modes)
Status	Categorical	Raw lead outcome/status (11 distinct values) — source for Lead_Category

A.2 Engineered Columns

Column	Derived From	Description
Lead_Category	Status	Target variable: High Potential / Low Potential (Section 7)
Lead_Category_Encoded	Lead_Category	Label-encoded target: High Potential = 0, Low Potential = 1
Created_Year	Created	Calendar year the lead was captured
Created_Month	Created	Calendar month the lead was captured (1–12)
Created_Day	Created	Day of month the lead was captured
Has_Mobile	Mobile	Binary flag — 1 if a usable mobile number was captured, else 0
Has_Email	EMAIL	Binary flag — 1 if a usable email address was captured, else 0
Source_*, Sales_Agent_*	Respective categorical	One-hot encoded indicator columns generated

Column	Derived From	Description
Location_*, Delivery_Mode_*	columns	for modelling (Section 9)

Appendix B: Consolidated Model Metrics

The following table consolidates every metric captured across baseline training, hyperparameter tuning, and overfitting diagnostics (Sections 10–12) into a single reference table for all six model configurations evaluated in this project.

Model	Train Acc.	CV Acc. (mean ± std)	Test Acc.	Precision	Recall	F1-Score	Train–Test Gap
Gradient Boosting	0.7473	0.7337 ± 0.0105	0.7204	0.7149	0.7204	0.7112	0.0270
KNN	0.7424	0.7023 ± 0.0107	0.6900	0.6821	0.6900	0.6810	0.0524
Logistic Regression	0.7084	0.7011 ± 0.0149	0.6813	0.6961	0.6813	0.6851	0.0271
Random Forest (Tuned)	0.7441	0.7003 ± 0.0058	0.6772	0.7022	0.6772	0.6816	0.0669
Random Forest	0.7131	0.6946 ± 0.0081	0.6631	0.6916	0.6631	0.6676	0.0500
Decision Tree	0.6705	0.6668 ± 0.0132	0.6368	0.6856	0.6368	0.6402	0.0337

Reading this table alongside Section 12, the recommended model (Gradient Boosting) is the only configuration that ranks in the top two on every single column simultaneously — test accuracy, precision, recall, F1-score, and generalization gap — which is why it was selected over models that scored marginally higher on any one individual metric.

Appendix C: Recommendations & Next Steps for FicZon

C.1 Immediate Actions

- Deploy the Gradient Boosting model as a scoring step immediately after lead capture, writing a High Potential / Low Potential (and ideally a probability score) back into the CRM record so sales agents see it before their first outreach.
- Route High Potential leads to agents with strong historical conversion performance in the same location/product combination, using the feature importance findings in Section 14 as a starting point for agent-territory alignment.
- Share the corrected Status → Lead_Category mapping (Section 7.3) with the sales operations team so that future manual status updates remain consistent with the categories the model was trained on.

C.2 Data Quality Recommendations

- Standardise the Status field at data-entry time (e.g. a fixed dropdown of the 11 known values) to prevent future case-sensitivity drift such as 'CONVERTED' vs. 'converted'.
- Capture Mobile and EMAIL validity at the point of entry rather than relying on downstream detection of placeholder values such as '#VALUE!'.
- Consider consolidating the long tail of Location values into a smaller number of standardised regions to reduce the one-hot feature space (currently contributing significantly to the 64-column feature matrix in Section 9.2).

C.3 Monitoring & Retraining

- Track live model accuracy and the High Potential / Low Potential split monthly; a material drift in either signals that retraining is due.
- Retrain on a rolling window (e.g. trailing 12 months) so the model adapts to new products, agents, and seasonal patterns identified in Section 5.6.
- Re-run the overfitting diagnostics in Section 12 after every retraining cycle, not just accuracy, so silent generalization regressions are caught early.

C.4 Future Enhancements

- Explore target encoding for Sales_Agent and Location as an alternative to one-hot encoding, to reduce dimensionality while preserving the strong geographic and agent-level signal found in Section 14.
- Incorporate additional CRM fields not present in the current extract (e.g. lead engagement history, product tier, deal size) if/when they become available, since Product_ID and Location currently carry a disproportionate share of the model's predictive power.

- Pilot the model on a held-out, live cohort of new leads before full rollout, comparing model-assisted prioritization against the current manual process on real conversion outcomes.